# Neural Posterior Estimation for the Chaotic Three-Body Problem

Taric Somani
21000530

William Pudupakkam
21003841

**AMATH 445: Scientific Machine Learning**
Instructor: Dr. Mohamed Kohandel
*University of Waterloo*
April 6, 2026

## 1 Introduction & Background

### 1.1 The Three-Body Problem

The three-body problem is a notorious problem in classical dynamics. It deals with the motion of three distinct bodies under the gravitational force as given by Newton's Law of Universal Gravitation. For a three-body system with masses $m_i$, and position vectors $\boldsymbol{r}$, the acceleration $\boldsymbol{a}$ of the $i$'th body is given by:

$$\boldsymbol{a}_i = \frac{\mathrm{d}^2\boldsymbol{r}}{\mathrm{d}t^2} = G\sum_{i\neq j} m_j \frac{\boldsymbol{r}_j - \boldsymbol{r}_i}{|\boldsymbol{r}_j - \boldsymbol{r}_i|^3} \tag{1.1}$$

where $G$ is the gravitational constant. Since these forces depend non-linearly on the inverse square of the distances between the bodies, the system is highly chaotic. The phase space is characterized by positive Lyapunov exponents, meaning that any microscopic perturbation in the initial state will cause the subsequent trajectories to diverge exponentially over time [1].

The three-body problem is, in general, not integrable and has no analytic closed-form solution, unlike its two-body counterpart, making the process of finding initial conditions computationally arduous [2].

### 1.2 Bayesian Inference

Given an observed sequence of spatial and velocity coordinates over time, the goal is to reconstruct the initial conditions:

$$\boldsymbol{\theta} = \left\{\boldsymbol{r}_{0,i}, \boldsymbol{v}_{0,i}, m_i\right\}_{i=1}^{3} \tag{1.2}$$

The continuous mapping from a trajectory to its initial conditions, although injective, is an ill-posed problem due to its chaotic nature. Trajectory observations are usually discretely sampled, leading to inherently uncertain data being used to formulate the initial conditions. This means that a conventional, deterministic neural network should not be used, as the initial conditions will be formed by a probability distribution over the manifold of plausible states (i.e., the set of all initial conditions consistent with the observation)[3].

When trained with a standard Mean Squared Error (MSE) loss, a deterministic model will mathematically converge to the conditional average of this complex distribution [4].

Because the phase space of the three-body problem is highly non-linear, it's possible for this mean to fall outside the manifold of valid solutions, landing in a region of near-zero probability. Consequently, integrating this averaged point estimate forward may not reconstruct the observed system. Therefore, a probabilistic Bayesian framework is needed to map the full geometry of the uncertainty.

### 1.3 Neural Posterior Estimation (NPE)

Neural Posterior Estimation (NPE) uses deep learning to construct an expressive, parameterized, surrogate model of the posterior distribution, $q_\psi(\boldsymbol{\theta}|\boldsymbol{X}) \approx p(\boldsymbol{\theta}|\boldsymbol{X})$. By simulating a dataset of prior-drawn parameters, and their corresponding observable tracks, a neural network is trained to learn the inverse mapping globally [5].

The architecture presented in this paper utilizes Normalizing Flows, which transform a simple base distribution, $q(z)$, into a more complex distribution,

$q(z')$, via a sequence of invertible transformations $z' = f(z)$, such that:

$$q(z') = q(z) \, |\det J_{f^{-1}}| \tag{1.3}$$

Where $J_{f^{-1}}$ is the Jacobian matrix of the inverse transformation evaluation at $z'$. If $q(z)$ can be rapidly evaluated and sampled from, and if $f$ is invertible and has a simple Jacobian determinant, then $q(z')$ can also be rapidly evaluated and sampled from [6].

This approach is heavily inspired by the DINGO (Deep INference for Gravitational-wave Observations) model [7], which successfully demonstrated that amortized neural inference can accurately reconstruct complex physical parameters in a short amount of time. By applying NPE to the planar three-body problem, the computational cost of inference is shifted entirely to the upfront training phase, allowing for near instantaneous parameter reconstruction.

# 2 Problem

To find the initial conditions from a discrete trajectory, one must use numerical integration methods. While the algorithmic complexity of evaluating the gravitational forces between $N$ bodies scales as $\mathcal{O}(N^2)$ [8], a computational bottleneck arises when the solver runs into chaotic "close encounters." As the distance between the bodies approaches zero during these encounters, the derivatives in the ODE approach infinity. To prevent the simulation from mathematically exploding, adaptive ODE solvers (such as Runge-Kutta methods) are forced to dynamically shrink their time steps ($\Delta t \to 0$) to maintain strict error tolerances. Consequently, the computational time required to force a deterministic integrator through a chaotic close encounter can be unpredictable [9].

The cost of integration can limit the usefulness of traditional inference methods such as Markov Chain Monte Carlo (MCMC). As MCMC relies on sequential, random-walk exploration, the expensive ODE solver will need to be run a large number of times. This renders MCMC highly inefficient for determining the posterior distribution.

In this paper, NPE is leveraged to infer the posterior distribution of the initial conditions. The trained model ingests a discretely sampled, 32-timestep track of spatial and velocity coordinates for each body $i$:

$$\boldsymbol{X}_{1:32} = \left\{ \left( \boldsymbol{r}_i^{(k)}, \boldsymbol{v}_i^{(k)}, t_k \right) \right\}_{k=1}^{32} \quad i \in \{1, 2, 3\} \tag{2.1}$$

and outputs the posterior distribution of the initial conditions $\boldsymbol{\theta}$ (Equation (1.2)). By replacing the sequential execution of ODE integrations with a single forward pass of a neural network, this architecture reduces the inference computation time significantly, solving the inverse problem's computational bottleneck.

# 3 Methodology

## 3.1 Data Generation

The first step of data generation required defining the prior probability distribution, $p(\theta)$, representing the bounds of the system before any observations are made. The initial conditions were randomly sampled from a uniform prior distribution. With setting $G = 1$, the masses were drawn from $m \sim \mathcal{U}(0.5, 2.5)$, spatial coordinates from $\boldsymbol{r}_0 \sim \mathcal{U}([-5, 5]^2)$, and velocities from $\boldsymbol{v}_0 \sim \mathcal{U}([-2, 2]^2)$.

For each drawn sample $\theta_j \sim p(\boldsymbol{\theta})$, the initial conditions were passed to SciPy's *solve_ivp* to simulate the evolution of the system [10]. To construct a comprehensive training set that mapped the entire phase space, the simulations were stratified into two regimes.

The first subset was comprised of $N_1 = 300,000$ trajectories sampled randomly from the prior to capture the generalized mechanics. However, the chaotic systems are dominated by strong interactions between bodies, which are a minority of the phase space volume, but represent the most difficult inverse mappings. To ensure the model adequately learned these volatile regions, a second subset of $N_2 = 150,000$ interacting trajectories was engineered by isolating initial conditions that guaranteed strong interactions within the integration window. This sampling strategy yielded a base dataset of $N_{\text{base}} = 450,000$ independent simulations.

To handle the volatile derivatives associated with the system of ODEs, the integrations were performed using the Dormand-Price 8(5,3) Runge-Kutta method (DOP853) [11].

In realistic scenarios, observers do not witness the initial conditions of a dynamic system and instead witness an evolved state at an arbitrary later epoch. To replicate this temporal displacement, the data generation protocol decoupled the system's ab-

solute initial state from the synthesized observation window.

During the forward integration, the DOP853 algorithm constructed a high-order continuous interpolating polynomial over the time span, $t \in [0, t_{\max}]$. Any trajectories that resulted in collisions before a minimum temporal threshold were discarded from the dataset. For all stable simulations, an observation window of $\Delta T_{\text{obs}}$ was uniformly sampled from the continuous trajectory, with the starting epoch defined as $t_{\text{start}} \sim \mathcal{U}(0, t_{\max} - \Delta T_{\text{obs}})$. The exact spatial and velocity coordinates were then evaluated at 32 equidistant timestamps across the observation window. The resulting vectors were then concatenated into the final $32 \times 13$ observation tensor $\boldsymbol{X}$.

Since the system of ODE's is injective, and hence deterministic, a perfect model would yield a Dirac-delta function likelihood, with an observation relating to a unique initial condition. However, as established by Judd & Smith (2001), real-world inverse problems have observational uncertainty, which expands a point solution into a manifold of possible states [3]. To force the network to learn this uncertainty, Gaussian noise was injected into the simulated trajectories. Since the features have different numerical ranges, the standard deviation of each feature was calculated across the temporal dimension:

$$\boldsymbol{\sigma}_{\textbf{feat.}} = \sqrt{\text{Var}(\boldsymbol{X}, \text{dim} = 0)} \in \mathbb{R}^{13} \tag{3.1}$$

The simulated error was defined as 0.5% of this natural variance. Noise was then added through:

$$\tilde{\boldsymbol{X}} = \boldsymbol{X} + \boldsymbol{Z} \odot 0.005\boldsymbol{\sigma}_{\textbf{feat.}} \tag{3.2}$$

where $\boldsymbol{Z} \sim \mathcal{N}(0, 1)$, and $\odot$ denotes the Hadamard product. The blurred trajectories $\tilde{\boldsymbol{X}}$ were then appended to the original dataset. The 450,000 original simulations provide the mathematical truth, forcing the model to learn the orbital dynamics of the system. Conversely, the 450,000 augmented simulations allow the network to become robust and generalizable. This strategy doubled the training volume, resulting in a final dataset of $N = 900,000$ samples.

## 3.2 Network Architecture

The inference process has two parameterized components: a temporal embedding network to compress the data, and a density-estimating Normalizing Flow to model the posterior probability distribution.

### 3.2.1 Embedding Network

The observation tensor $\boldsymbol{X} \in \mathbb{R}^{32\times13}$ contains a highly correlated, multi-dimensional time series. Feeding this directly into the density estimator would greatly increase the training time due to the high dimensionality of the data. To resolve this, an embedding network was utilized to turn the trajectory into a 128-dimensional summary vector. The embedding network was implemented using Pytorch [12].

First, a Multi-Layer Perceptron processes each time step. This expands the 13-dimensional state into a 64-dimensional representation. The extended time series is then processed by a 1-Dimensional Convolutional Neural Network (1D-CNN). Parallel 1D convolutions with kernel sizes of $k = 3$ and zero-padding were used to expand the channel depth to 128. Spatial dropout with $p = 0.2$ is applied between convolutional layers. An Adaptive Average Pooling layer then marginalizes the temporal dimension, collapsing the 32-step evolution into a single vector. The pooled tensor is flattened and passed through a final layer, yielding the 128-dimensional summary vector $\boldsymbol{h} = f_\varphi(\boldsymbol{X})$, where:

$$f_\varphi : \mathbb{R}^{32\times13} \to \mathbb{R}^{128} \tag{3.3}$$

### 3.2.2 Normalizing Flows

To approximate the posterior distribution $p(\boldsymbol{\theta} \mid \boldsymbol{X})$, Normalizing Flows were utilized. These are a class of generative models that transform a base distribution, such as a Gaussian, into a complex distribution using a sequence of invertible and differentiable transformations [6]. The flows were constructed using Neural Spline Flows (NSFs). NSFs utilize rational-quadratic splines to warp the probability space, allowing the network to model the non-linear

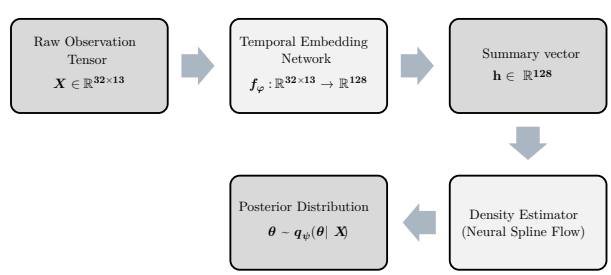


**Figure 1:** Data flow of the Neural Posterior Estimator.

posterior [13]. Batch normalization and dropout were both applied during the training process. The resulting surrogate posterior is defined as:

$$q_\psi(\theta|\boldsymbol{h}) \approx p(\theta|\boldsymbol{X}) \tag{3.4}$$

The architecture was compiled and trained using the Simulation-Based Inference (SBI) framework [14]. Prior to training, the dataset was standardized to zero mean and unit variance to account for the different numerical scales of the input parameters.

The embedding network weights ($\varphi$) and the normalizing flow weights ($\psi$) were jointly optimized. The objective of the network is to maximize the expected log-likelihood of the true initial conditions given the observed trajectories. Thus, the network was trained to minimize the Negative Log-Likelihood loss function over each batch of size $B = 4096$:

$$\mathcal{L}(\varphi, \psi) = -\frac{1}{B}\sum_{i=1}^{B} \log q_\psi\left(\theta^{(i)}|f_\varphi\left(\boldsymbol{X}^{(i)}\right)\right) \tag{3.5}$$

where $\theta^{(i)}$ are the true initial conditions, and $f_\varphi\left(\boldsymbol{X}^{(i)}\right)$ is the 128 dimensional vector produced by the embedding network.

### 3.2.3 Training Protocol

To prevent the model from memorizing the data, a training protocol was enforced. The training loop was bounded by a hard computational ceiling of 1,000 epochs and governed by an early-stopping mechanism automatically implemented by the SBI package. A 10% fraction of the dataset was held out as a validation set and monitored continuously. Training was stopped when the validation log-probability failed to improve for 20 consecutive epochs.

All neural network training procedures were executed on a single NVIDIA GeForce GTX 1080ti GPU. Under this configuration, the training loop converged after 472 epochs, with a total training time of 6 hours and 10 minutes.

# 4 Results

To evaluate the robustness and accuracy of the model, testing was conducted on a held-out set of 50,000 trajectories sampled from a subset specifically consisting of interacting trajectories.

## 4.1 Global Performance

The global performance of the model was first evaluated using point-estimate metrics, calculated between the true initial conditions and the expected values of the predicted posteriors. Using the posterior mean as a point estimate, the model reported an RMSE of 0.0162 and an MAE of 0.0014 on the test set. The larger RMSE tells us that even though most predictions are very close to the true parameters, a smaller number of cases gain much larger errors, which skews the RMSE. It's important to remember, however, that the primary goal of the framework is to capture the full probability distribution mapping the geometry of the uncertainty.

We used the Testing Accuracy with Randomized Probabilities (TARP) method to measure expected coverage [15] (Figure 2). The empirical coverage curve forms an S-shape pattern, reflecting a calibration mismatch that varies with credibility level. For credibility levels $\alpha < 0.5$, the empirical coverage falls below the ideal diagonal, indicating that the posterior is too narrow relative to the true posterior, showing local overconfidence. Conversely, for credibility levels $\alpha > 0.5$, the coverage rises above the ideal diagonal, indicating large credible regions contain the true parameters more frequently than they should. This tendency towards overcoverage is also observed by the 90% credibility level achieving 98.45% empirical coverage, which exceeds the nominal rate, as well as the area-to-curve (ATC) statistic of 1.177, compared to an ideal value of 1.0. Nonetheless, a Kolmogorov-Smirnov (KS) test yields $p = 0.963$ indicating that these deviations from perfect calibration are not statistically significant.

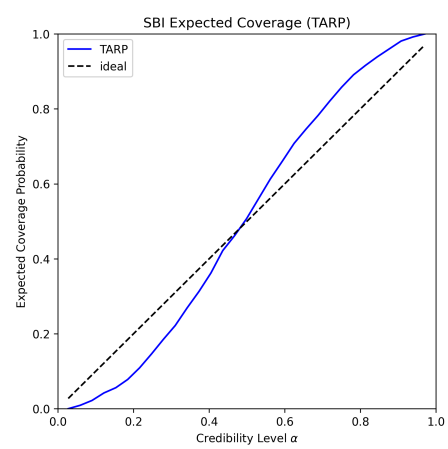


**Figure 2:** Testing Accuracy with Randomized Probabilities (TARP) showing the expected coverage of the surrogate posterior.

As shown by the TARP analysis, the posteriors exhibit underconfidence, characterized by heavier tails than the true posterior. While this represents a calibration limitation, underconfidence may be preferable to overconfidence for this chaotic inverse problem. In well-calibrated inference, the true posterior should already be broad due to the exponential sensitivity of chaotic trajectories and observational noise, however, the observed posterior suggests that the surrogate is overestimating the true posterior width, though this ensures conservative coverage of viable parameter regions. In contrast, an overconfident approximation would compound the inference challenge by artificially narrowing an already-difficult-to-estimate posterior, potentially excluding physically plausible initial conditions.

Furthermore, Simulation-Based Calibration (SBC) was employed to evaluate the uniformity of the posterior ranks [16]. The SBC rank histograms reveal mostly acceptable calibration with some underconfidence (Figure 3). While most rank statistics remain within the 99% confidence bands (e.g., $x_1$), dimensions such as 3, 7, 10, 12, and 15 exhibit slight central peaks (e.g., $v_{y2}$), which is a characteristic of overdispersed posteriors. This confirms the TARP findings of underconfidence. The full array of SBC rank histograms for all 15 dimensions is provided in Appendix A (Figure 8).

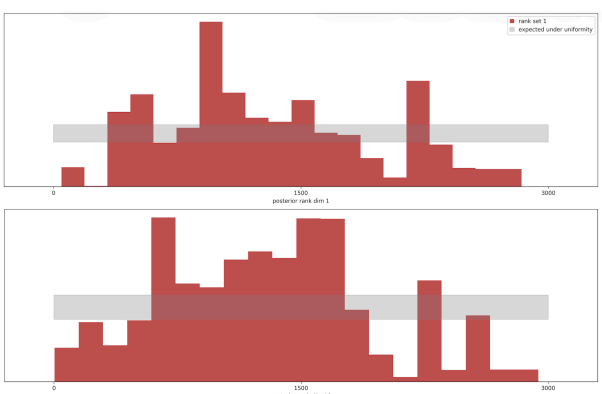


**Figure 3:** Simulation-Based Calibration (SBC) Rank Histograms for $x_1$ and $v_{y2}$, respectively.

Overall, the surrogate demonstrates computational reliability without any catastrophic calibration failures, making it suitable for inference while acknowledging that uncertainty estimates are conservatively wide.

## 4.2 Local Performance

Beyond global calibration, the local quality of the posterior distributions was analyzed. We employed the Local 2-Classifier Sample Test (L-C2ST) to compare the NPE-generated posteriors against reference posteriors on a per-observation basis [17]. The observed L-C2ST test statistic was 0.0102 ($p = 0.14$), as illustrated in Figure 4.

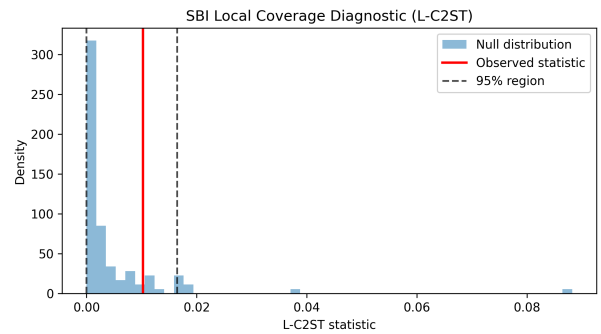


**Figure 4:** Histogram of the Local 2-Classifier Sample Test (L-C2ST) statistic for a representative observation. The test statistic measures the deviation from random chance (0.5 accuracy) in distinguishing NPE samples from reference posterior samples. Values near zero indicate indistinguishable distributions.

With a p-value of 0.14, we fail to reject the null hypothesis that the two distributions are identical, indicating that the generated posteriors are consistent with reference posteriors at the local level.

The necessity of this probabilistic approach is evident when examining challenging trajectories. For certain observations, several inferred parameters, such as velocity components (such as $v_{x2}$ and $v_{y2}$), can exhibit skewed and multi-modal distributions (Figure 5). While not all observations produce such complex posteriors, the existence of these cases motivates the use of conditional density estimation.The complete set of posterior marginal distributions for all parameters can be found in Appendix A (Figure 9).

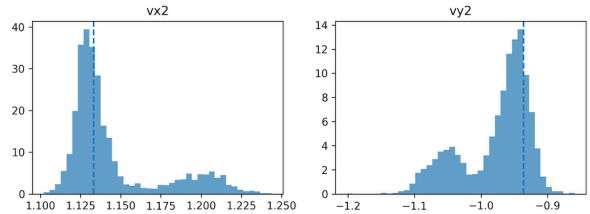


**Figure 5:** Sample posterior marginal distributions for inferred initial $v_{x2}$ and $v_{y2}$.

Had a traditional deterministic neural network been trained using an MSE loss, it would have converged to the mean of the distribution (i.e. a lower probability valley between these peaks), resulting in a failure to reconstruct the system dynamics.

Finally, posterior predictive checks were performed by sampling initial conditions from the inferred posterior and integrating them forward using the DOP853 solver. For stable trajectories (Figure 6), the simulations stayed close to the true observations, suggesting that the parameter estimates were accurate. Conversely, for highly chaotic trajectories (Figure 7), the small uncertainties in the posterior spread out into a wide range of possible future paths. This suggests that the model can represent unpredictability in the system's chaotic regimes rather than collapsing to an invalid point estimate.

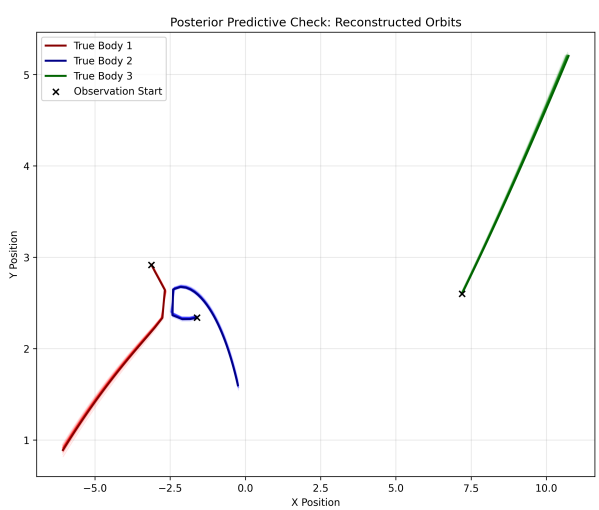


**Figure 6:** Sample posterior predictive check of a stable orbital regime.

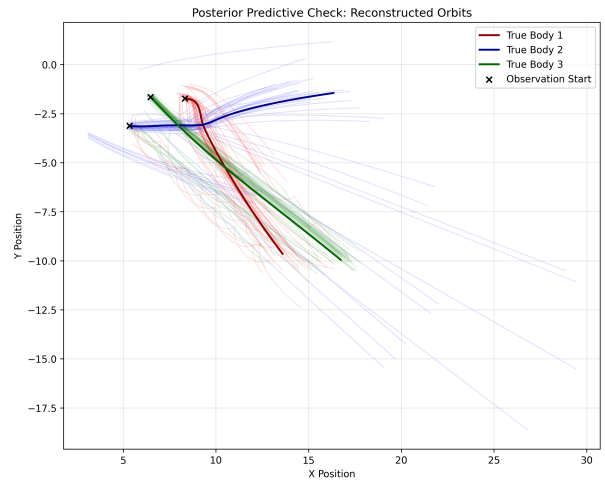


**Figure 7:** Sample posterior predictive check of a highly chaotic orbital regime.

# 5 Discussion

The application of NPE to the chaotic planar three-body problem demonstrates that amortized neural inference can overcome the computational bottlenecks associated with traditional Bayesian methods. Shifting the computational burden of numerical integration to the upfront training phase allows the trained architecture to approximate the complex posterior distribution of initial conditions in a fraction of a second. In addition to achieving sufficient global coverage, the model captures the multi-modal and non-linear geometries of the local posteriors, though calibration diagnostics indicate general underconfidence.

To determine the practical limits of this architecture, the model was tested against a "stress-test dataset" comprising of highly volatile trajectories with extreme sensitivities to initial conditions. In these highly chaotic regimes, microscopic uncertainties in the posterior lead to divergent futures. Crucially, rather than collapsing to an invalid, deterministic point estimate, the NPE framework successfully bounds this manifold of possibilities, generating a conservative probabilistic map of the system's inherent unpredictability following close encounters.

The evaluation also revealed specific sources of error and model quirks. As identified in the TARP expected coverage diagnostics, the surrogate posteriors exhibit an S-shaped calibration profile: they are slightly overconfident near the mode but safely conservative in the tails. While a heavy-tailed posterior is an advantageous failure mode for predicting

chaotic dynamics, other architectural constraints persist. Using adaptive average pooling across time in the embedding network might indirectly truncate the precise movements needed to accurately detect the complexity involved with near-collisions. Furthermore, challenging posterior characteristics may exceed the expressive capacity of the rational-quadratic splines in the Normalizing Flows, leading to poor approximation quality.

Future work will focus on expanding the physical realism and generalizability of the framework. A primary progression is to upgrade the physical simulation from two-dimensions to a full three-dimensional spatial domain. Furthermore, rather than using a hard-coded gravitational constant $G$, the architecture can be modified to treat $G$ as a free parameter to be inferred alongside the state variables, accommodating observations of systems with unknown mass scales. Finally, there is potential to scale this methodology beyond the three-body problem to $N$-body systems. This expansion should involve more advanced embedding architectures, such as Graph Neural Networks, to handle the increase in phase space volume and interaction complexity. All code is available at https://github.com/7ar1c/three_body_npe.

## 6 Acknowledgements

The authors acknowledge the use of Google's Gemini AI during the preparation of this project. The tool was used as a programming and typesetting assistant to debug code and copy-edit the authors' original drafts for clarity. All methodologies, data analyses, and discussions are the original work of the authors.

## References


[1] M. Valtonen and H. Karttunen, *The Three-Body Problem*. Cambridge University Press, 2006.

[2] T. Gowers, J. Barrow-Green, and I. Leader, *The Princeton Companion to Mathematics*, Illustrated edition. USA: Princeton University Press, 2008.

[3] K. Judd and L. A. Smith, "Indistinguishable states I: perfect model scenario," *Physica D: Nonlinear Phenomena*, vol. 151, no. 2–4, pp. 125–141, 2001.

[4] C. M. Bishop, "Mixture density networks," Aston University, Working Paper 4288, 1994.

[5] D. S. Greenberg, M. Nonnenmacher, and J. H. Macke, "Automatic Posterior Transformation for Likelihood-Free Inference." [Online]. Available: https://arxiv.org/abs/1905.07488

[6] D. Rezende and S. Mohamed, "Variational Inference with Normalizing Flows," in *Proceedings of the 32nd International Conference on Machine Learning*, F. Bach and D. Blei, Eds., in Proceedings of Machine Learning Research, vol. 37. Lille, France: PMLR, 2015, pp. 1530–1538. [Online]. Available: https://proceedings.mlr.press/v37/rezende15.html

[7] M. Dax, S. R. Green, J. Gair, J. H. Macke, A. Buonanno, and B. Schölkopf, "Real-Time Gravitational Wave Science with Neural Posterior Estimation," *Phys. Rev. Lett.*, vol. 127, no. 24, p. 241103, Dec. 2021, doi: 10.1103/PhysRevLett.127.241103.

[8] J. Barnes and P. Hut, "A hierarchical O(N log N) force-calculation algorithm," *Nature*, vol. 324, no. 6096, pp. 446–449, 1986.

[9] W. Dehnen and J. I. Read, "N-body simulations of gravitational dynamics," *The European Physical Journal Plus*, vol. 126, no. 5, May 2011, doi: 10.1140/epjp/i2011-11055-3.

[10] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[11] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I*, 2nd ed. Berlin, Heidelberg: Springer-Verlag, 1993.

[12] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library." [Online]. Available: https://arxiv.org/abs/1912.01703

[13] C. Durkan, A. Bekasov, I. Murray, and G. Papamakarios, "Neural Spline Flows." [Online]. Available: https://arxiv.org/abs/1906.04032

[14] A. Tejero-Cantero *et al.*, "sbi: A toolkit for simulation-based inference," *Journal of Open Source Software*, vol. 5, no. 52, p. 2505, 2020, doi: 10.21105/joss.02505.

[15] P. Lemos, A. Coogan, Y. Hezaveh, and L. Perreault-Levasseur, "Sampling-Based Accuracy Testing of Posterior Estimators for General Inference." [Online]. Available: https://arxiv.org/abs/2302.03026

[16] S. Talts, M. Betancourt, D. Simpson, A. Vehtari, and A. Gelman, "Validating Bayesian Inference Algorithms with Simulation-Based Calibration." [Online]. Available: https://arxiv.org/abs/1804.06788

[17] J. Linhart, A. Gramfort, and P. L. C. Rodrigues, "L-C2ST: Local Diagnostics for Posterior Approximations in Simulation-Based Inference." [Online]. Available: https://arxiv.org/abs/2306.03580

# A. Appendix

## A.1. Full Simulation-Based Calibration Results

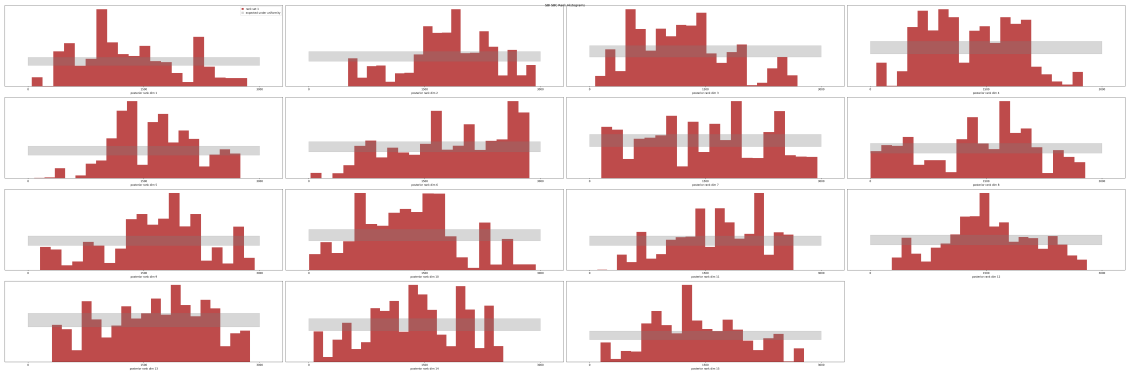

**Figure 8:** Full array of Simulation-Based Calibration (SBC) Rank Histograms for all 15 inferred initial conditions and masses.

## A.2. Full Posterior Marginal Distributions

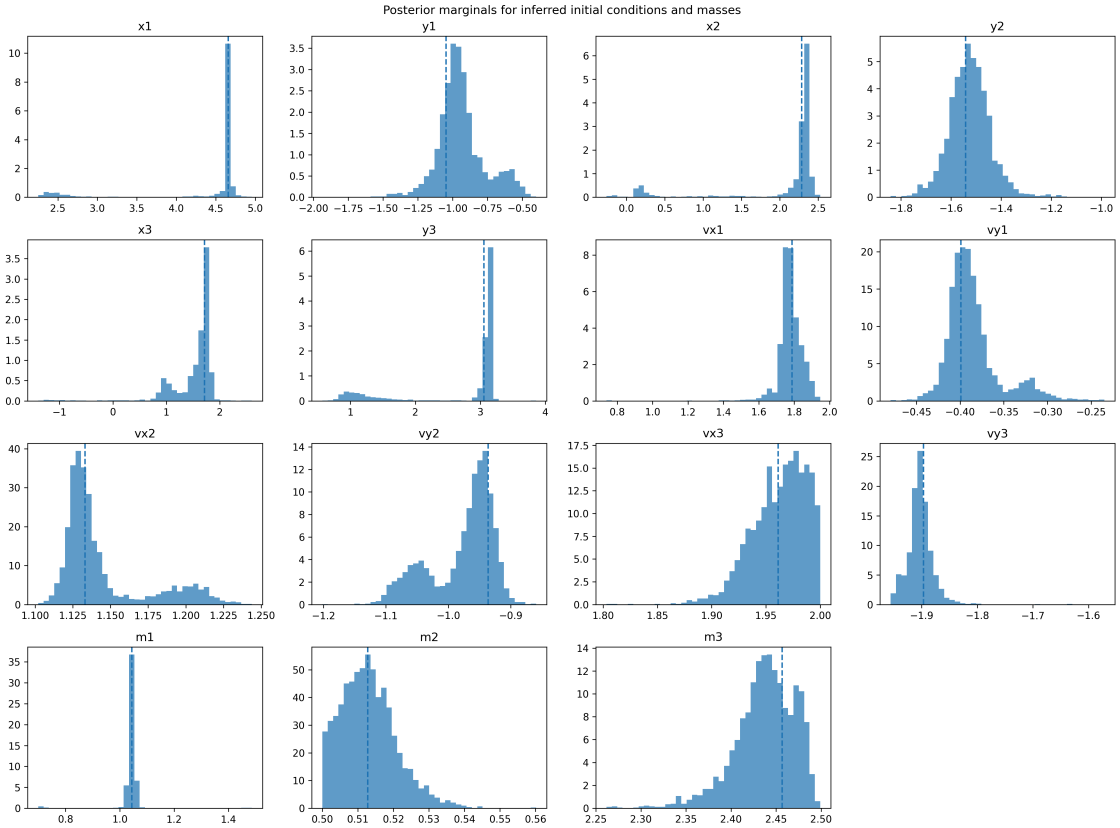


**Figure 9:** Complete set of posterior marginal distributions for all inferred parameters.